



Guía de Evaluación: Semana 2 - IA-Native Quality & TDD (Mid Level)

Taller: Calidad Nativa con IA, TDD Avanzado y Estrategia de Pruebas

Tema: "La IA genera la implementación, el humano garantiza la intención."

1. Objetivo de la Evaluación

Validar la evolución del estudiante de un "Desarrollador que Testea" a un "**Ingeniero de Calidad IA-Native**". Se evaluará su capacidad para usar la IA no solo para escribir código rápido, sino para anticipar fallos mediante TDD (Red-Green-Refactor). Buscamos confirmar que entienden que en la era de la IA, la única verdad inmutable y red de seguridad es una suite de pruebas robusta que **Verifique** la lógica técnica y **Valide** las reglas de negocio.

2. El Reto: "La Suite Inquebrantable"

Los equipos deben implementar una nueva funcionalidad de negocio compleja o refactorizar una existente en su Arquitectura Hexagonal. Sin embargo, **la evaluación no se centrará en el código de producción, sino en la red de seguridad construida alrededor de él.**

Deben garantizar "Zero Errors" (cero fallos de regresión en pruebas heredadas y nuevas) y demostrar que la IA fue guiada *a partir de las pruebas*, y no al revés.

3. Conceptos Clave a Evaluar (El "Human Check" de la Calidad)

Para aprobar, el equipo debe evidenciar dominio sobre estos conceptos durante su defensa y en el código:

- **TDD (Red-Green-Refactor):** Evidencia de que los casos de prueba de borde se generaron *antes* de la implementación.
- **Verificar vs. Validar:** Distinción clara en los tests entre verificar que la sintaxis funciona (Unit/Integration) y validar que el negocio esté protegido (ej. evitar el saldo negativo).
- **QA vs. Testing:** Explicar su *estrategia* general (QA) en contraposición a la simple ejecución de scripts (Testing).

4. Entregables Obligatorios

1. **Código Fuente:** Repositorio actualizado donde las pruebas y el código de producción conviven respetando la Arquitectura Hexagonal.

2. **TESTING_STRATEGY.md:** Documento que define la estrategia de QA, diferenciando qué probaron para *verificar* y qué para *validar*.
3. **Reporte Automatizado de Pruebas:** Archivo exportado o configurado en el pipeline (ej. reporte de Jest/Mocha en Node, JUnit/Jacoco en Java, PyTest en Python, o xUnit en .NET).
4. **Integridad de la Suite (Zero Errors):** Captura de pantalla en el nuevo archivo evidenciando que el 90% de las pruebas pasan exitosamente.

5. Rúbrica de Evaluación

Criterio	Nivel Experto (5.0) - Senior Track	Nivel Competente (3.5) - Mid Track	Nivel Insuficiente (2.0) - Junior Track
Integridad y Eficacia de la Suite	100% de pruebas en verde. La suite atrapa inyecciones de errores intencionales (mutaciones). Los mocks en pruebas unitarias son precisos y aíslan perfectamente.	100% de pruebas en verde. Sin embargo, algunas pruebas son frágiles o prueban detalles de implementación en lugar de comportamientos.	Fallos en las pruebas (rojo), fallos de regresión, o pruebas que pasan siempre porque no evalúan aserciones reales ("Teatro de la Calidad").
Ciclo TDD IA-Native	El historial de Git y las explicaciones demuestran un ciclo claro: 1) IA genera tests (RED). 2) IA/Humano implementan mínimo para pasar (GREEN). 3) Refactor seguro.	Hay tests sólidos, pero se evidencia que algunos fueron escritos <i>después</i> de la implementación (After-thought) solo para cumplir el requisito.	No hay evidencia de TDD. Los tests se agregaron al final, pidiéndole a la IA "Hazle pruebas a este archivo".
Reportes Automatizados	Configuración impecable del generador de reportes (ej. Jacoco, Jest coverage). El reporte es legible,	El reporte automatizado existe y se genera correctamente, mostrando los tests que pasaron y fallaron, pero	No hay reporte automatizado. Dependen de leer el output de la consola manualmente.

	muestra métricas de cobertura lógica (branches) y se ejecuta con un solo comando.	carece de análisis de cobertura de casos borde.	
Verificar vs. Validar	Diferencian claramente pruebas unitarias puras (Verificar) de pruebas que validan reglas críticas de negocio simulando el caso de uso completo (Validar).	Entienden los conceptos teóricamente, pero en el código la mayoría de las pruebas solo <i>verifican</i> flujos felices sin validar restricciones de negocio.	Confunden los conceptos. Los tests prueban constructores, getters/setters o funciones triviales generadas por la IA sin impacto de negocio.
Human Check: Comprensión	Dominio total. Pueden explicar línea por línea qué hace un test complejo generado por la IA y cómo garantiza la continuidad si se borrara el código de producción.	Entienden el objetivo general del test, pero dudan al explicar configuraciones de Mocks/Stubs introducidas por la IA.	No logran explicar cómo funcionan las aserciones (expects / asserts) generadas por la IA en sus propios tests.

6. Formato de Presentación (Viernes PM)

Los equipos no harán una presentación tradicional. Se someterán a una auditoría técnica en vivo:

1. **Defensa de Estrategia: Verificar vs Validar:** El equipo debe abrir su TESTING_STRATEGY.md y mostrar en código:
 - Un test que *Verifique* la arquitectura (ej. que se llame correctamente a una interfaz de un puerto).
 - Un test que *Valide* el negocio (ej. asegurar que el "Saldo Fantasma" negativo no sea posible).
2. **El "Human Check" de Comprensión:** El instructor elegirá un test de integración o unitario aleatorio del repositorio y le pedirá a un miembro del equipo que explique exactamente qué dependencias se están mockeando y por qué.